Yeditepe University
Department of Computer Engineering

CSE 232
Systems Programming

Lecture Notes #1

INTRODUCTION

1. What is Systems Programming?

A computer system consists of
- internal hardware (e.g. microprocessor), external hardware (e.g. I/O devices) and
- system programs

System Programs:

- make a computer system easy to use for a nonexpert
- make it possible for the resources of the system to be used efficiently

<table>
<tr><td colspan="4">Application programs</td></tr>
<tr><td colspan="2">Compilers</td><td colspan="1">Assemblers</td><td>Macroprocessor</td></tr>
<tr><td>Loaders</td><td>Text Editors</td><td>Searching/Sorting</td><td>Debuggers</td></tr>
<tr><td colspan="4">Operating Systems</td></tr>
<tr><td colspan="4">Monitors</td></tr>
<tr><td colspan="4">I/O routines</td></tr>
<tr><td colspan="4">Hardware</td></tr>
</table>

Figure 1. A simple computer system

I/O routines: manipulate interfaces and send or receive single characters from I/O devices

Monitors: simple software used to control the computer system. Some example commands may be: ENTER, DISPLAY, RUN, EDIT, LOAD, STORE

Operating systems: in addition to the functions of a monitor, operating systems perform the management of the file system, management of the concurrent execution of tasks, management of terminals, etc.

Assemblers: translate a program written in assembly language into machine language

Text Editors: allow text (programs, etc) to be edited on-line

Compilers: translate programs written in a high-level language into machine language

Interpreters: analyze what each line in the program is asking for and then does it immediately

Loaders: place programs into memory and prepare them for execution

Linkers: bind subroutines and library functions together

2. Number System

- Binary numbers (Base 2): 0,1
- Octal numbers (Base 8): 0,1,2,3,4,5,6,7
- Decimal numbers (Base 10): 0,1,2,3,4,5,6,7,8,9
- Hexadecimal numbers (Base 16): 0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F

Example:

$(314)_{10}=(100111010)_2=(472)_8=(13A)_{16}$

$(0.3)_{10}= (0.\overline{01001})_2=(0.\overline{23146})_8=(0.4\overline{C})_{16}$

Complements

- *1's complement*: To find the 1's complement of a binary number, replace all 0's by 1's and all 1's by 0's
- 2*'s complement*: To find the 2's complement of a binary number, add 1 to its 1's complement

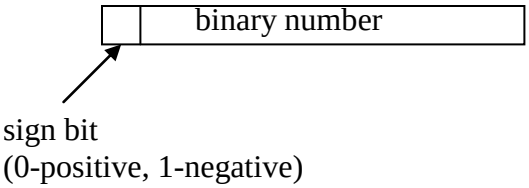
Example:

binary number:	00110101
1's complement:	11001010
2's complement:	11001011

Number Representation

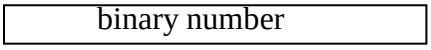
Representation of Integers

1. Signed-magnitude representation

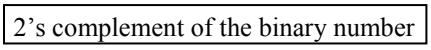


2. 2's complement representation

   Positive numbers:



   Negative numbers:



Example:

Representation of $(21)_{10}$ with 8 bits:

Signed-magnitude representation:	00010101
2's complement representation:	00010101

Representation of $(-21)_{10}$ with 8 bits:

Signed-magnitude representation:	10010101
2's complement representation:	11101011

Representation of Floating-point Numbers

$$\text{mantissa} \times \text{radix}^{\text{exponent}}$$

Example:

Representation of $(1001.11)_2$ with 8-bit mantissa and 4-bit exponent:

$(1001.11)_2 = +\underbrace{(.100111)_2}_{\text{mantissa (normalized)}} \times 2^{\overbrace{+4}^{\text{exponent}}} =$

mantissa	exponent
0 1 0 0 1 1 1 0	0 1 0 0

Mantissa is represented as signed-magnitude, exponent is represented as 2's complement.

2. Systems Environment

2.1. Basic Computer Hardware

Von Neumann Architecture

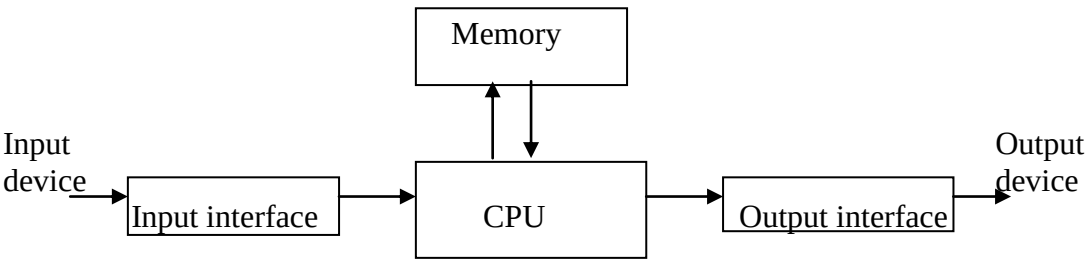


Figure 2. *Basic Computer Architecture*

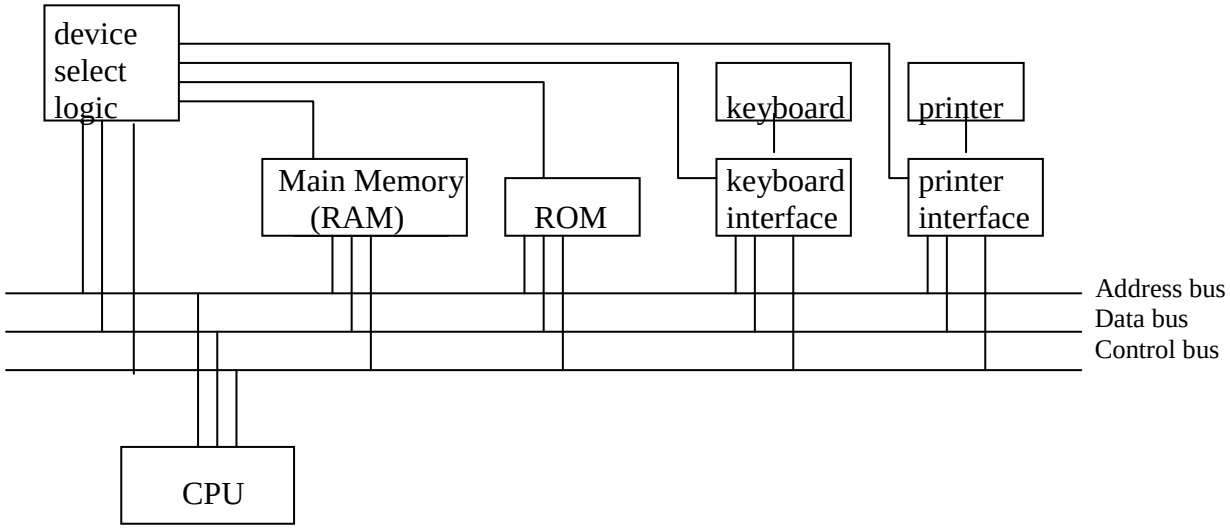


Figure 3. *Interconnection of computer components*

Central Processing Unit (CPU)

CPU controls the operation of the computer and processes data.

Major components of CPU are:

- **Arithmetic Logic Unit (ALU):** performs arithmetic and logic operations
- **Control Unit:** controls the operation of CPU
- **Registers:** Storage space inside the CPU
- Interconnection mechanism for communication among ALU, control unit and registers

CPU Registers:

PC (program counter): contains the memory address of the next instruction to be executed
IR (instruction register): contains the current instruction
MAR (memory address register): contains the memory address where the data will be stored or read.
MBR (memory buffer register): contains a copy of the designated memory location specified by the MAR after a read operation or the new contents of the memory location prior to a write operation
AC (accumulator): contains the data that is used in arithmetic or logic operations
General purpose registers: used by the programmer as temporary storage area

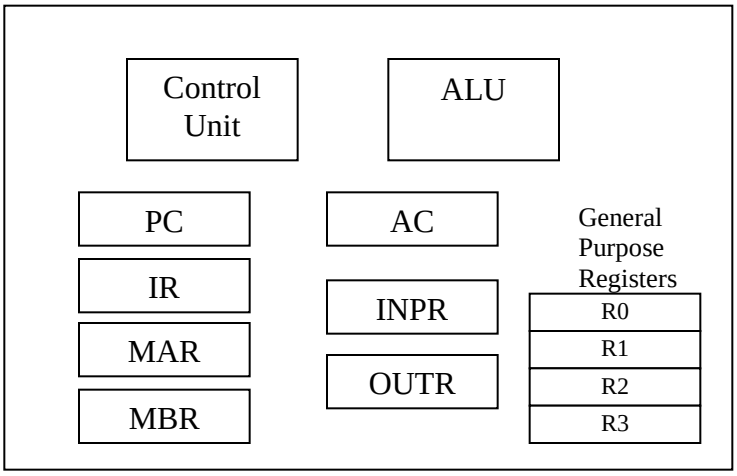


Figure 4. Inside the *CPU*

Main Memory

Instructions and data are stored in the main memory.
A memory **word** is a group of bits that are written in or read out of the memory as a unit. The addressable unit in main memory is a word.

Example: A memory module has word length of 32-bits and addressed by 16 bits. The size of this memory is 256 Kbytes.

System Bus

System bus is the communication path that connects CPU, memory and I/O units.

Bus lines can be classified into three functional groups:

- data lines
- address lines
- control lines

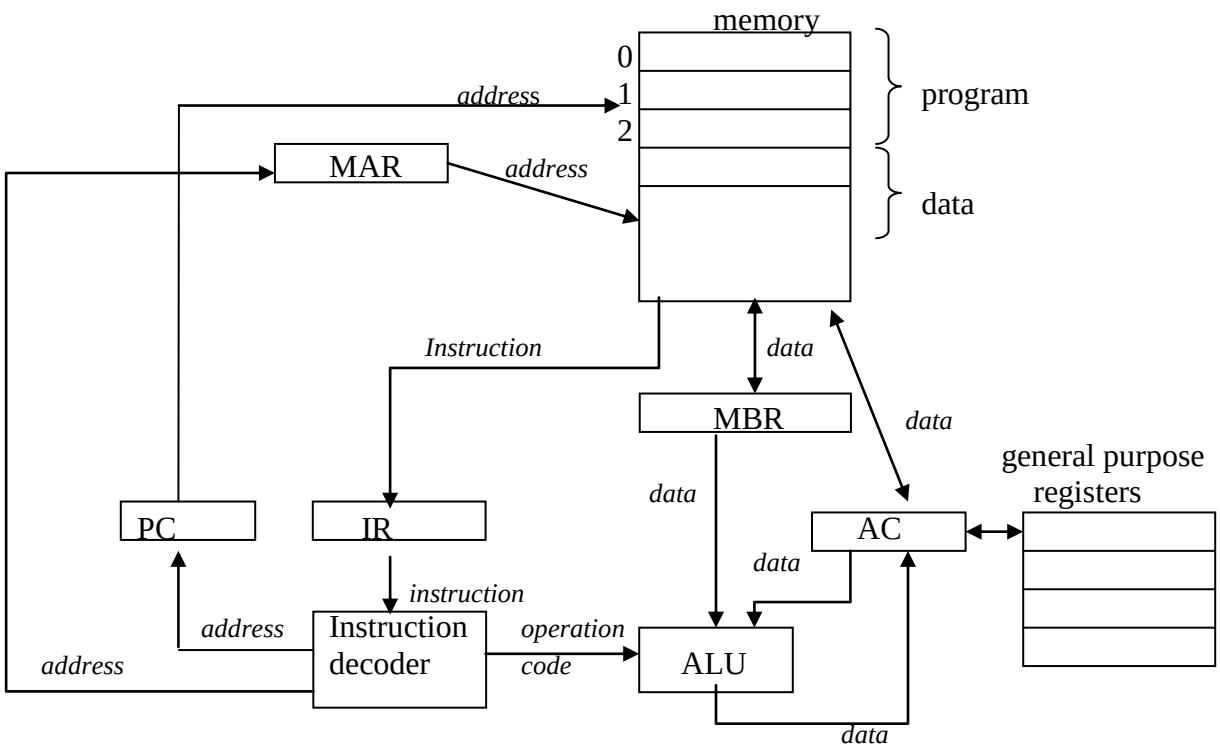


Figure 5. *Information flow between the components of a computer system*

3. Instruction Execution Cycle

1. **Fetch instruction:** Reading the instruction from memory into IR
2. **Decode instruction:** Determining the opcode and the operand specifiers
3. **Calculate effective address and fetch operands:** Calculating the address of the operands and fetching the operands from memory into registers
4. **Execute instruction:** Performing the indicated instruction
5. **Write result:** Storing the result in the memory

Example*:* Execution of ADD instruction

Op-code	memory address	memory address
	(addr1)	(addr2)

ADD 10A, 10C

M[10A] ← M[10A] + M[10C]

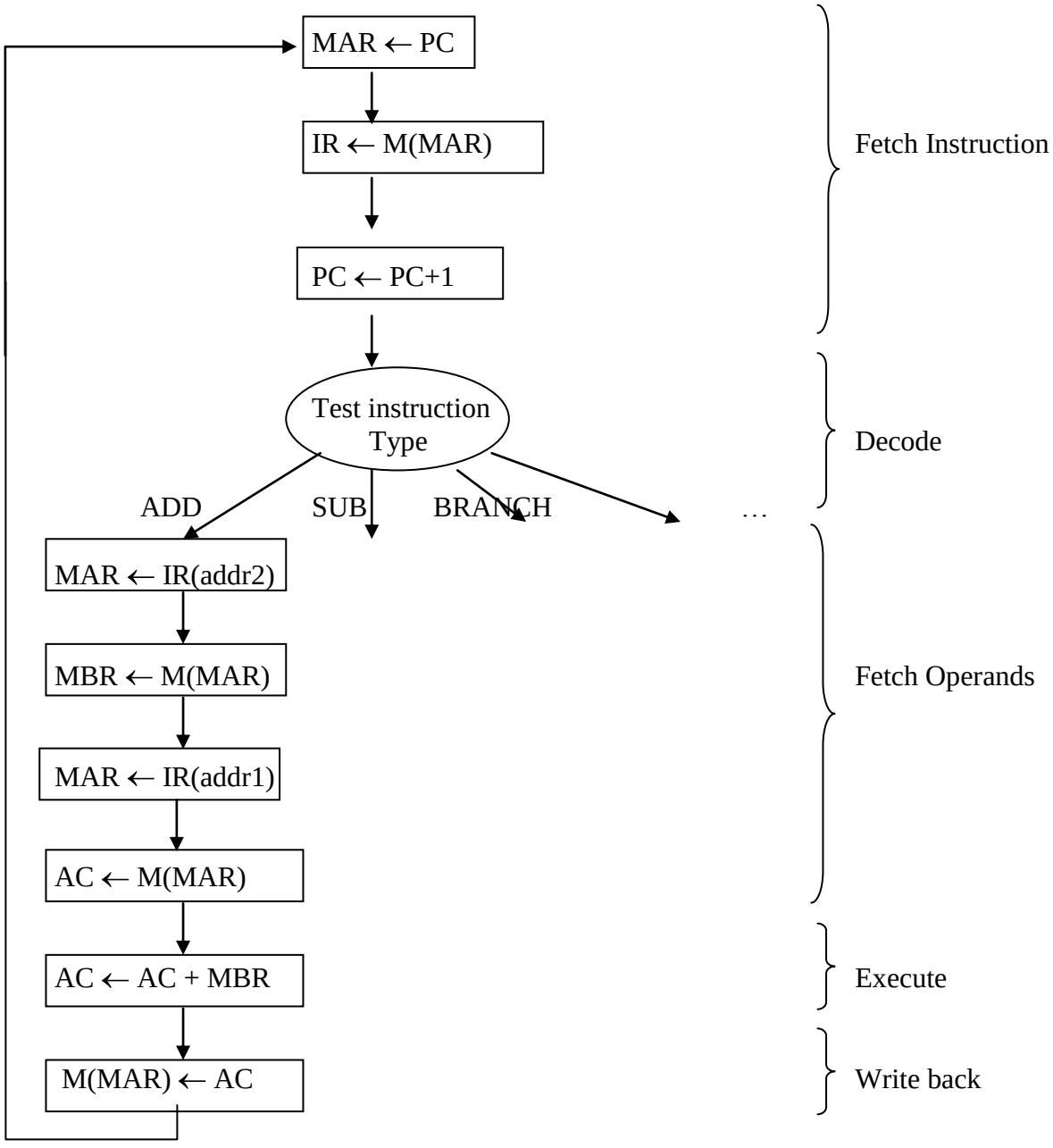


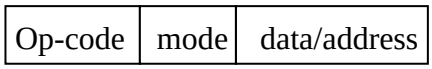
Figure 6. *Flowchart for the execution of ADD instruction*

4. Instruction Formats

Instruction length should either be equal to the memory transfer length or a multiple of it.

Three types of information in an instruction:

1. *Operation code (opcode)*: specifies the operation to be performed
2. *Address/data*: designates a memory address or a processor register or an operand
3. *Addressing Mode*: specifies the way the operand or effective address is determined



Example:
```
ADD   R1, R2
CLR   A
```

Operations

Operation type	Examples
Arithmetic and logical	Integer arithmetic and logical operations: add, subt., and, or
Data transfer	Load/store, move instructions
Control	Branch, jump, procedure call and return, traps
System	Operating system call, virtual memory management inst.
Floating point	Floating point operations: add, multiply
Decimal	Decimal: add, multiply, decimal to char conversion
String	String: move, compare, search

Figure 7. *Categories of instruction operators*

The first three (arithmetic and logic, data transfer, and control) are basic operations which exist in all instruction sets.

Operands

Number of operands in an instruction:

- 3-operand format (a result and 2 source operands)
- 2-operand format (one of the operands is both the result and the source operand)
- 1-operand format (the operand is both the result and the source operand)
- 0-operand format (no operands)

Operand use in instructions:

- register-register (all operands are in registers),
- register-memory (some operand are in memory and some operands are in registers),
- memory-memory (all operands are in memory)

Addressing Modes:

Addressing mode of an operation show how the CPU will access the operands.

Effective Address (EA): The actual memory address specified by an addressing mode

Addressing Modes:

- *Implied addressing:* The operand is implied in the opcode
- *Immediate addressing:* The operand is specified in the instruction itself
- *Direct addressing:* Effective address is equal to the address part of the instruction
- *Indirect addressing:* The address field of the instruction gives the address where the effective address is stored in memory
- *Register addressing:* The operands are in registers
- *Register indirect:* Instruction specifies a register whose contents give the address of the operand in memory
- *Indexed addressing:* The content of the index register is added to the address part of the instruction to obtain the effective address
- *Base register addressing:* the content of a base register is added to the address part of the instruction to obtain the effective address
- *Auto-increment or auto-decrement:* similar to register indirect except that the register is incremented or decremented after its value is used to access memory
- *Relative addressing:* The content of the program counter is added to the address part of the instruction to obtain the effective address

Addressing Mode	**Example instruction**	**Meaning**	**When used**
Immediate or Literal	ADD #3	AC←AC+3	For constants
Direct or absolute	ADD 100	AC←AC+M[100]	Static data access
Indirect or deferred	ADD @(100)	AC←AC+M[M[100]]	100 contains the address of the operand
Register	ADD R4, R3	R4←R4+R3	When a value is in a register
Register deferred or indirect	ADD R4, (R1)	R4←R4+M[R1]	Accessing using a pointer-computed address
Indexed	ADD 50,X ADD R3, (R1+R2)	AC←AC+M[50+IX] R3←R3+M[R1+R2]	Useful in array addressing
Displacement or based	Add R4,100(R1)	R4←R4+M[100+R1]	Accessing local variables
Auto-increment	Add R1, (R2)+	R1←R1+M[R2] R2←R2+d	Useful for stepping thru. arrays within a loop. R2, start of array
Auto-decrement	Add R1, -(R2)	R2←R2-d R1←R1+M[R2]	Same use as auto-incr., they are also used in stack operations
Scaled or Index	Add R1,100(R2)[R3]	R1←R1+ M[100+R2+R3*d]	Used to index arrays. any base addr. mode

Figure 8. *Selection of addressing modes with examples, meaning and usage*

Example:

PC=200
R1=400 (processor register)
IX=100 (index register)

Address	Memory
…	
200	LDA 500
202	next instruction
…	
3FE	450
400	700
…	
500	800
…	
600	900
…	
702	325
…	
800	300

Addressing modes:

Immediate:	AC=500	
Direct:	AC=800	(EA=500)
Indirect:	AC=300	(EA=800)
Relative:	AC=325	(EA=500+202)
Indexed:	AC=900	(EA=IX+500)
Register:	AC=400	
Register Indirect:	AC=700	(EA=400)

Instruction Set Design

Instruction set depends on the computer architecture. We can distinguish 3 main instruction set architectures:

1. *Accumulator architecture:* one operand is implicitly the accumulator (e.g., PDP8, Motorola 6809).
   Ex: C=A+B

```
LOAD    A
ADD     B
STORE   C
```

2. *Stack architecture:* the operands are implicitly on the top of stack (e.g., B5500, HP 3000/70)
   Ex: C=A+B

```
PUSH  A
PUSH  B
ADD
POP   C
```

3. *General purpose register architectures:* have only explicit operands-either registers or memory locations (e.g., IBM 360, DEC VAX).
   Ex: C=A+B

```
LOAD    R2, A
ADD     R2, B
STORE   C, R2
```